RAPPORT DE PROJET CAPSTONE EN SCIENCES DE DONNEES

SUMMARY

- Introduction
- Data collecting and data wrangling methodology
- EDA and interactive visual analytics methodology
- Predictive analysis methodology
- EDA with visualization results
- EDA with SQL results
- Interactive map results using Folium
- Ploty DASH dashboard results
- Predictive analysis (classification) results
- Conclusion

INTRODUCTION

Le présent rapport expose la méthodologie de collecte et de nettoyage des données, l'analyse exploratoire des données, l'apprentissage automatique et la visualisation des données.

Les résultats de notre travail sont présentés sous formes de graphiques, de cartes et de tableaux de bord.

METHODOLOGIE DE COLLECTE ET DE PREPARATION DE DONNEES

- Identifier les données nécessaires pour votre étude
- Identifier et acquérir les bonnes sources d'information
- Eliminer les doublons
- Traiter les valeurs manquantes
- Traiter les valeurs aberrantes
- Garantir la qualité, la fiabilité et la structure des données

COLLECTE DES DONNEES

Task 1: Request and parse the SpaceX launch data using the GET request

To make the requested JSON results more consistent, we will use the following static response object for this project:

[39] ✓ 0s

```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.clo
```

We should see that the request was successfull with the 200 status response code

[40] ✓ 0s

```
response.status_code
```

```
200
```

COLLECTE DES DONNEES

[54]
✓ 0s

```
# Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = data[data['BoosterVersion'] != 'Falcon 1']
```

Now that we have removed some values we should reset the FlgihtNumber column

[55]
✓ 0s

```
data_falcon9.loc[:,'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outco
4	1	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	No No
5	2	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	No No
6	3	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	No No

NETOYAGE DES DONNEES

TASK 1: Calculate the number of launches on each site

The data contains several Space X launch facilities: Cape Canaveral Space Launch Complex 40 **VAFB SLC 4E** , Vandenberg Air Force Base Space Launch Complex 4E **(SLC-4E)**, Kennedy Space Center Launch Complex 39A **KSC LC 39A** .The location of each Launch Is placed in the column `LaunchSite`

Next, let's see the number of launches for each site.

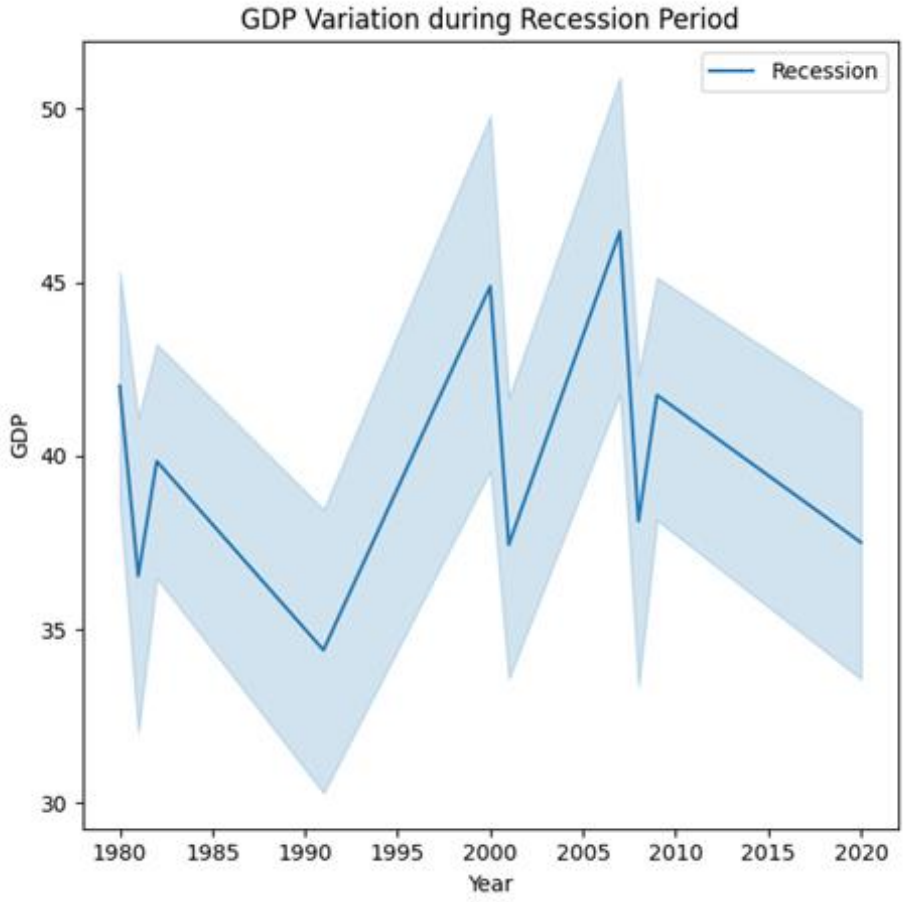
Use the method `value_counts()` on the column `LaunchSite` to determine the number of launches on each site:

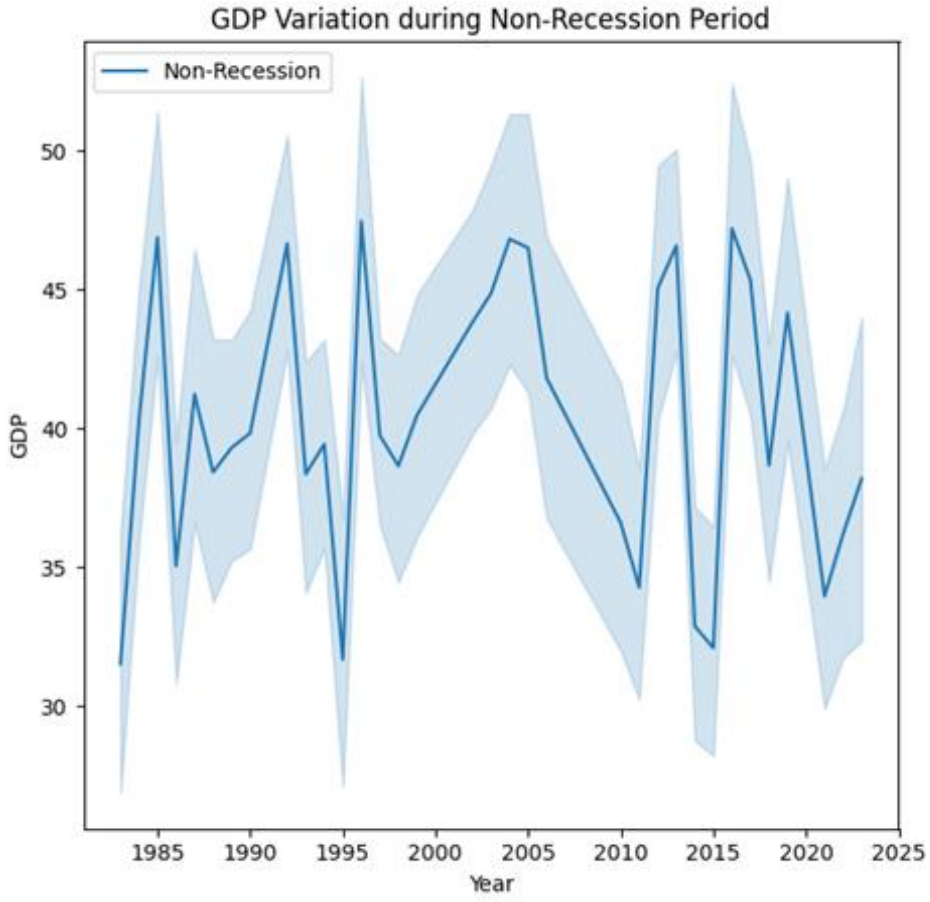
```
df['LaunchSite'].value_counts()
```

LaunchSite	count
CCAFS SLC 40	55
KSC LC 39A	22
VAFB SLC 4E	13

dtype: int64

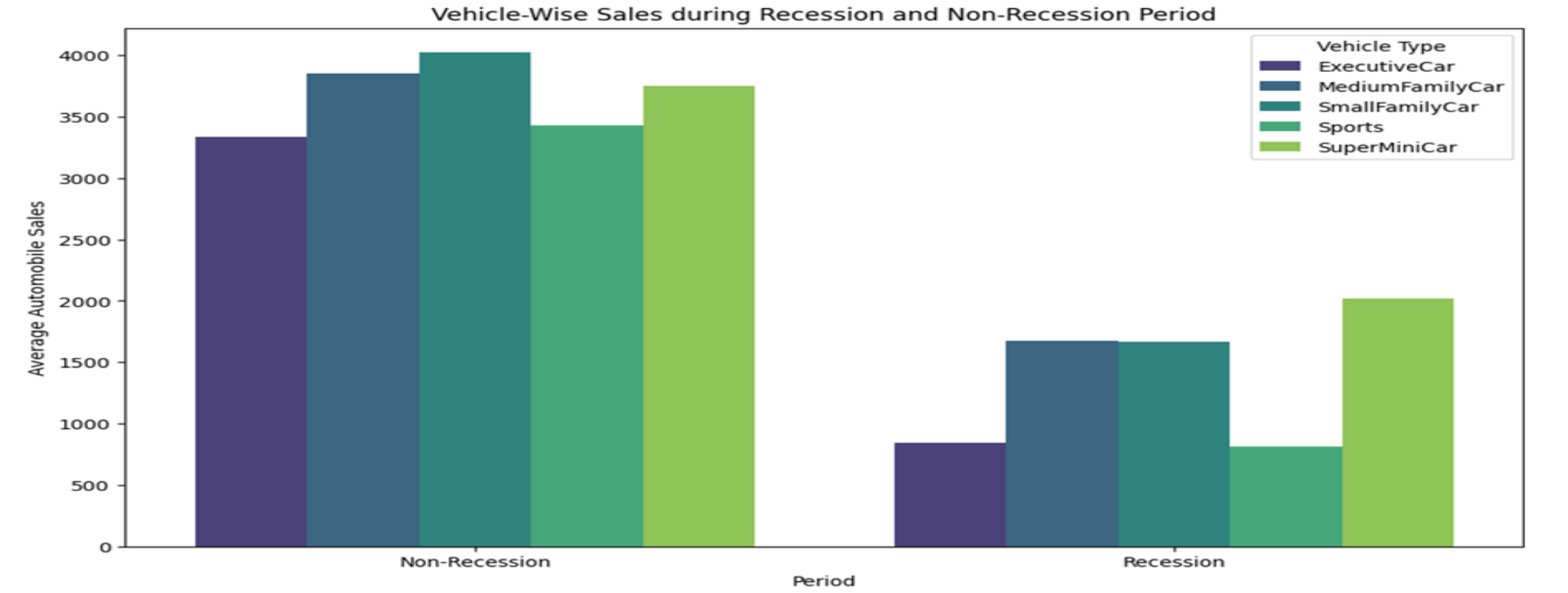
VISUALISATION DES DONNES





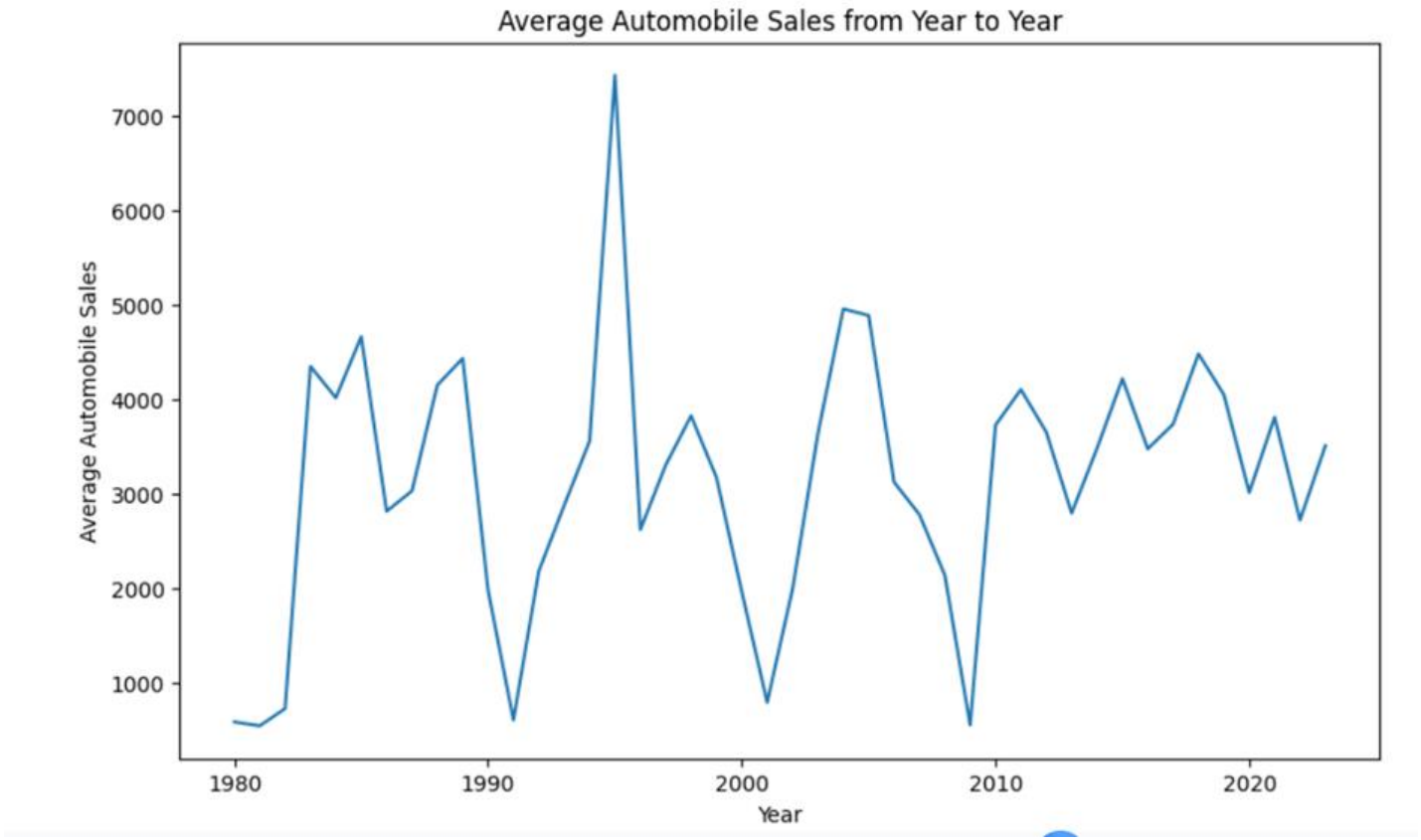
VISUALISATION DES DONNEES

```
plt.ylabel('Average Automobile Sales')
plt.title('Vehicle-Wise Sales during Recession and Non-Recession Period')
plt.legend(title='Vehicle Type')
plt.show()
```

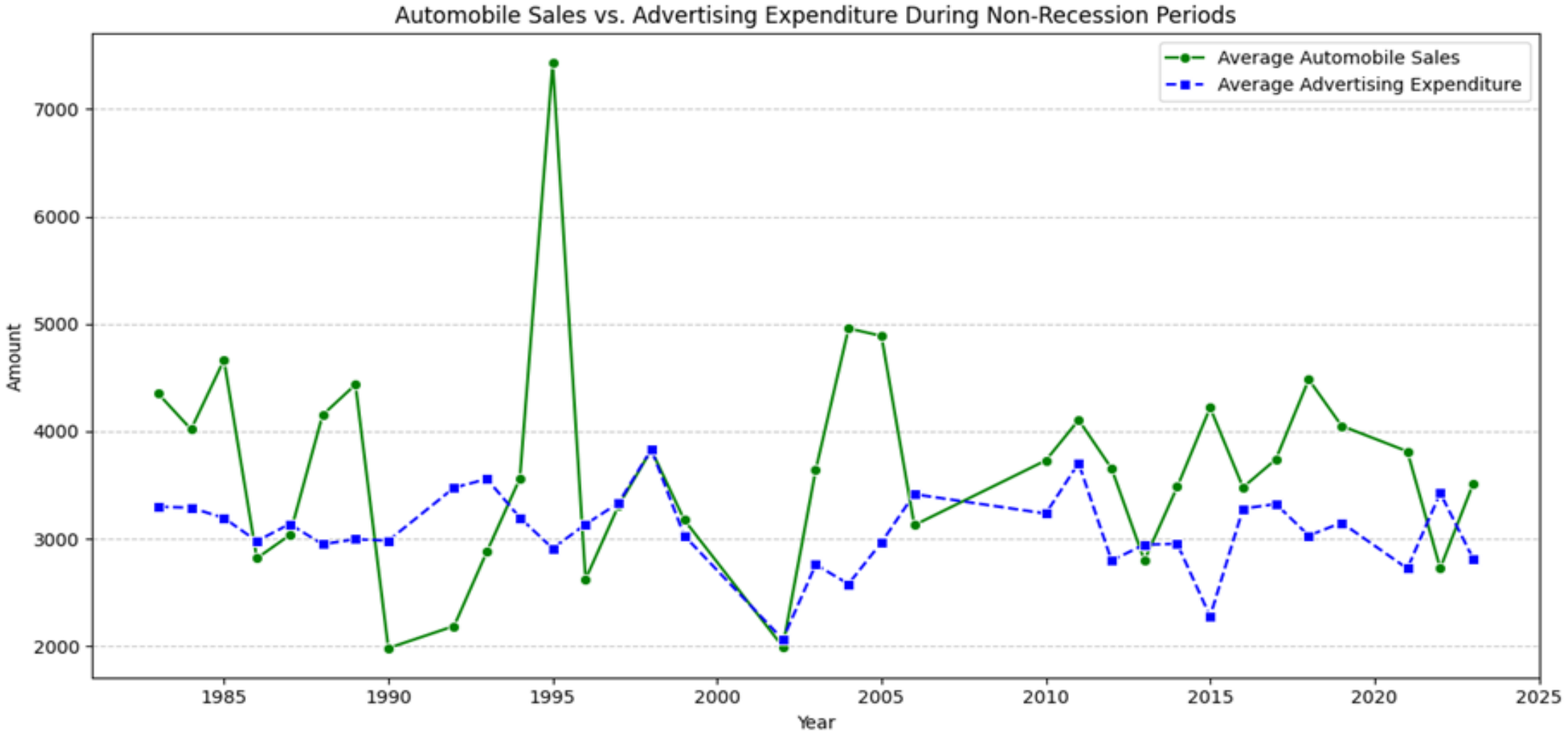


VISUALISATION DES DONNEES

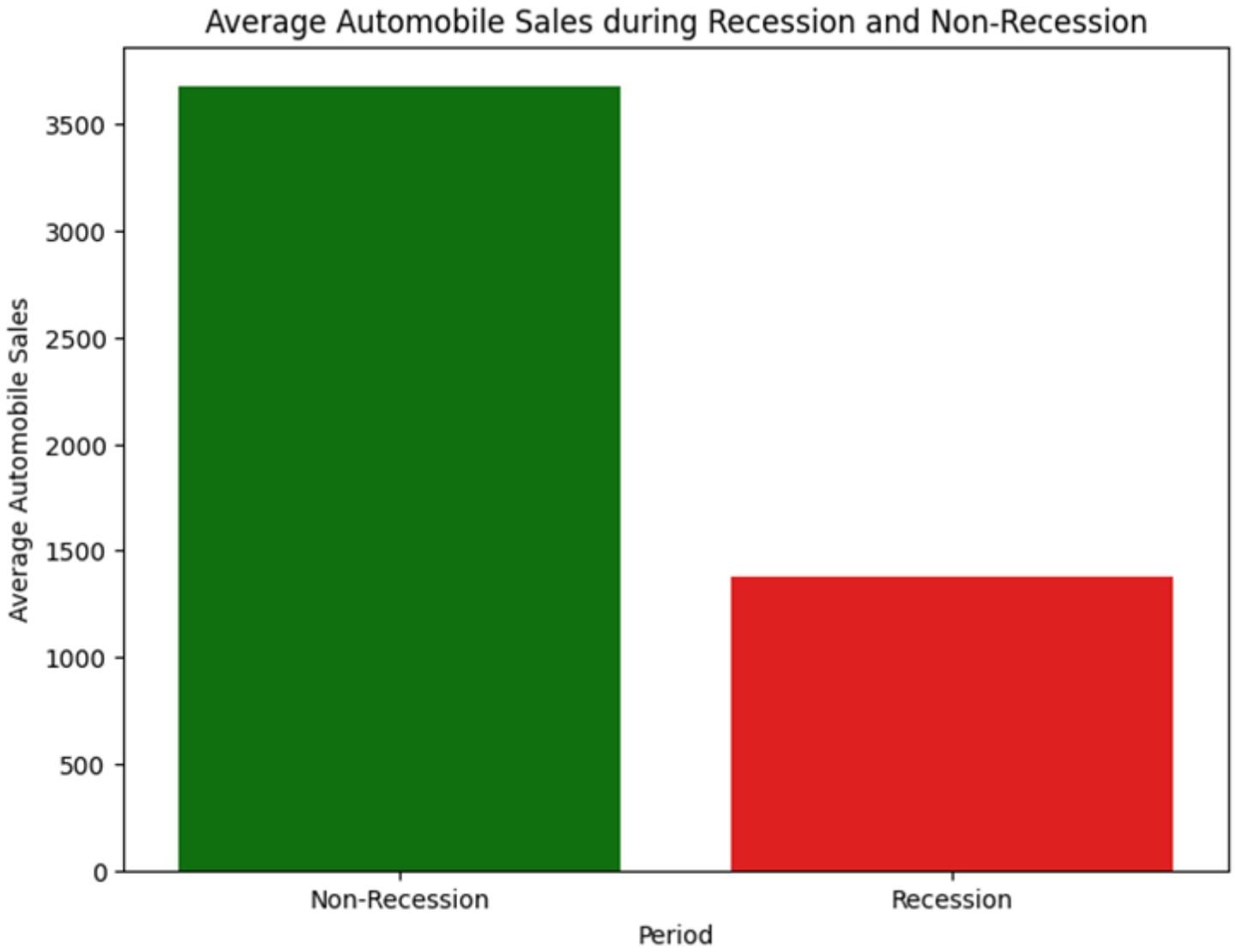
```
Data downloaded and read into a dataframe!
```



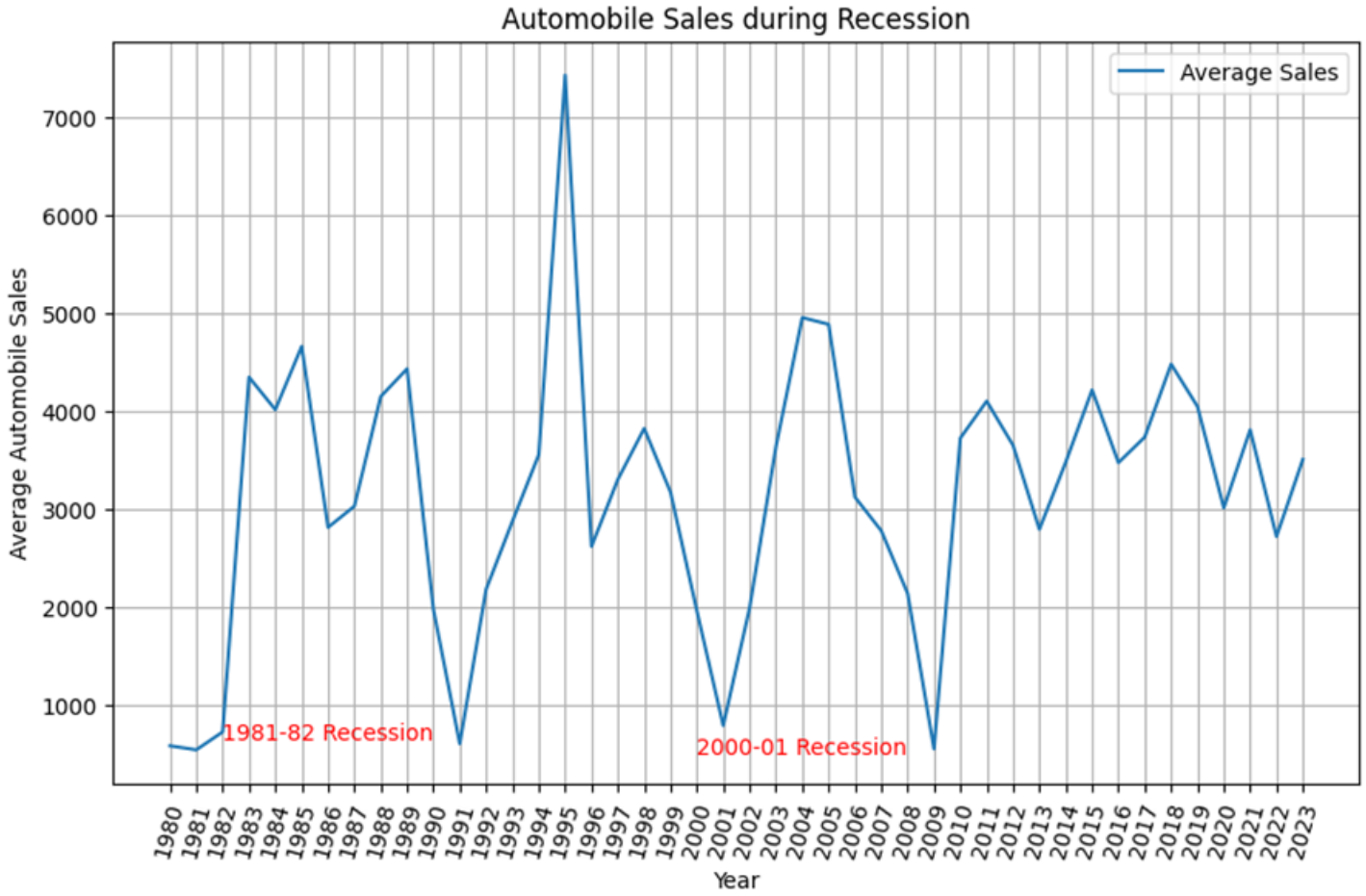
VISUALISATION DES DONNEES



VISUALISATION DES DONNEES

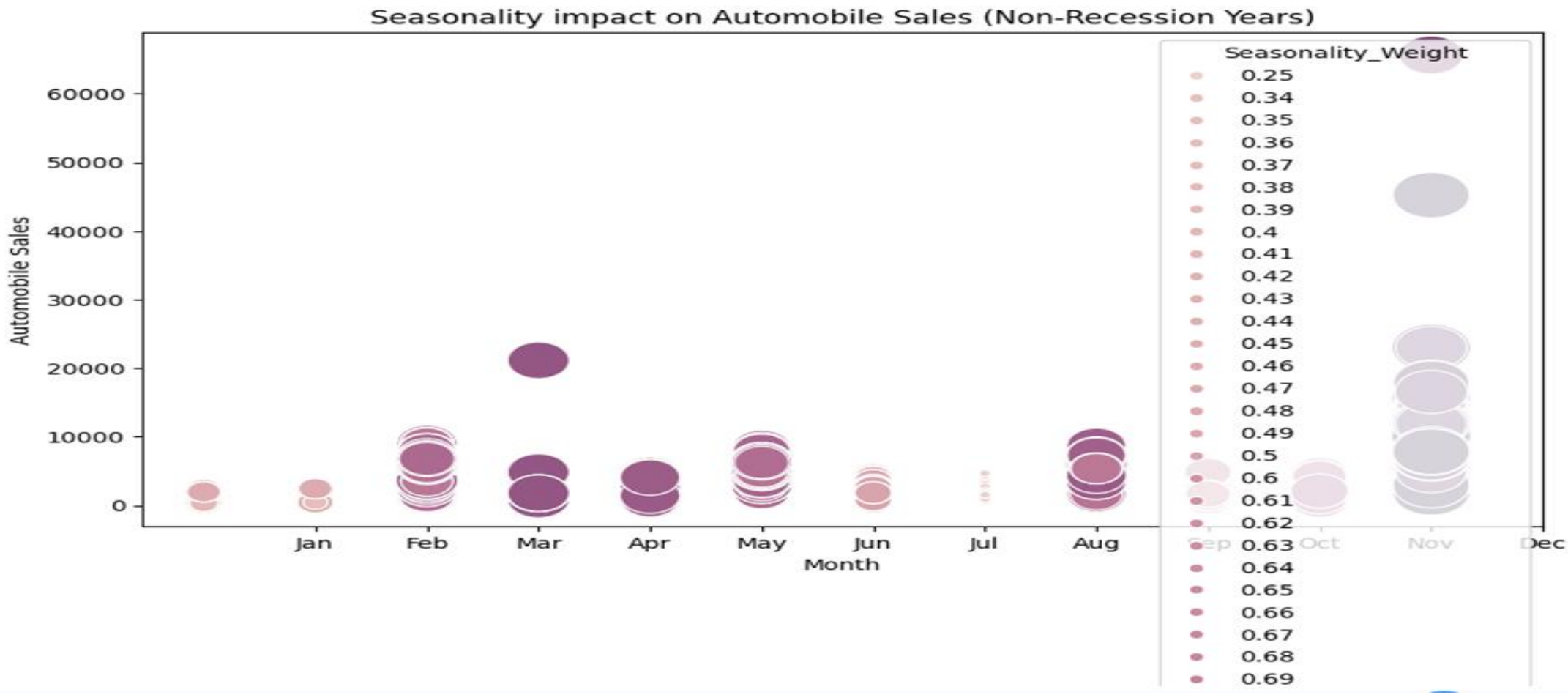


VISUALISATION DES DONNEES



VISUALISATION DES DONNEES

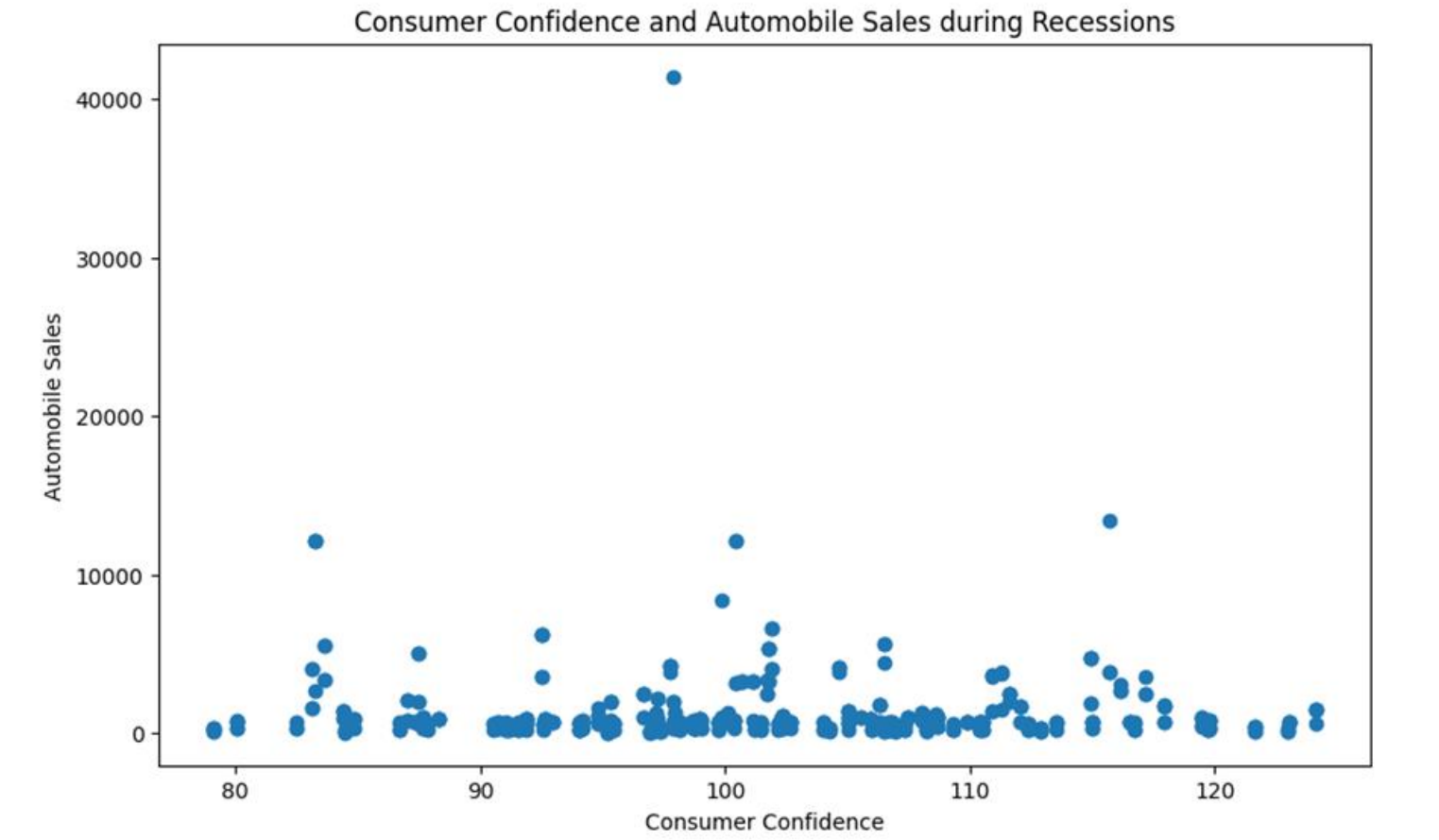
```
plt.title('Seasonality impact on Automobile Sales (Non-Recession Years)')
plt.xticks(range(1, 13), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
plt.show()
```



VISUALISATION DES DONNEES

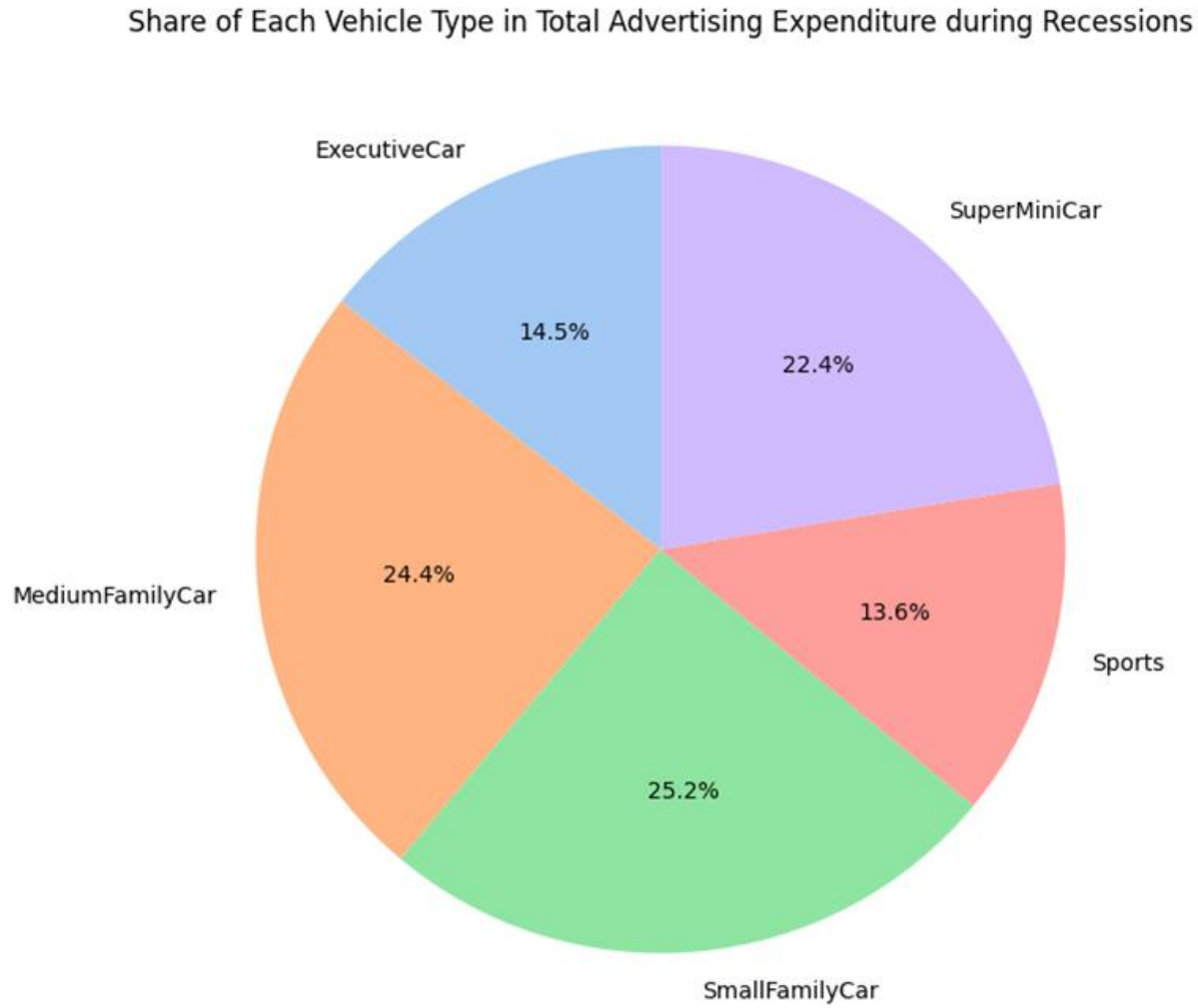
```
rec_data = df[df['Recession'] == 1]
plt.figure(figsize=(10, 6))
plt.scatter(rec_data['Consumer_Confidence'], rec_data['Automobile_Sales'])

plt.xlabel('Consumer Confidence')
plt.ylabel('Automobile Sales')
plt.title('Consumer Confidence and Automobile Sales during Recessions')
plt.show()
```



VISUALISATION DES DONNEES

```
plt.title('Share of Each Vehicle Type in Total Advertising Expenditure during Recessions')
plt.ylabel('') # Hide the default y-label
plt.show()
```



VISUALISATION DES DONNEES_SQL

Task 1

Display the names of the unique launch sites in the space mission

```
%sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE;
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

VISUALISATION DES DONNEES_SQL

Task 2

Display 5 records where launch sites begin with the string 'CCA'

```
%sql SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

VISUALISATION DES DONNEES_SQL

Display the total payload mass carried by boosters launched by NASA (CRS)

```
%sql SELECT SUM(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Customer = 'NASA (CRS)';
```

```
 * sqlite:///my_data1.db
Done.
```

SUM(PAYLOAD_MASS__KG_)
45596

VISUALISATION DES DONNEES_SQL

Display average payload mass carried by booster version F9 v1.1

```
%sql SELECT AVG(PAYLOAD_MASS__KG_) FROM SPACEXTABLE WHERE Booster_Ve
```

```
 * sqlite:///my_data1.db
Done.
```

AVG(PAYLOAD_MASS__KG_)
2928.4

Task 5

List the date when the first succesful landing outcome in ground pad was ach

Hint:Use min function

```
%sql SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome = 'Succ
```

```
 * sqlite:///my_data1.db
Done.
```

MIN(Date)
2015-12-22

VISUALISATION DES DONNEES_SQL

List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000

```
%sql SELECT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 6000;
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Task 7

List the total number of successful and failure mission outcomes

```
%sql SELECT Mission_Outcome, COUNT(Mission_Outcome) FROM SPACEXTABLE GROUP BY Mission_Outcome;
```

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	COUNT(Mission_Outcome)
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

ANALYSE VISUELLE_CARTE FOLIUM

```
import pandas as pd
import wget

# Download and read the `spacex_launch_geo.csv`
spacex_csv_file = wget.download('https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/spacex_launch_geo.csv')
spacex_df=pd.read_csv(spacex_csv_file)
```

Now, you can take a look at what are the coordinates for each site.

```
# Select relevant sub-columns: `Launch Site`, `Lat(Latitude)`, `Long(Longitude)`, `class`
spacex_df = spacex_df[['Launch Site', 'Lat', 'Long', 'class']]
launch_sites_df = spacex_df.groupby(['Launch Site'], as_index=False).first()
launch_sites_df = launch_sites_df[['Launch Site', 'Lat', 'Long']]
launch_sites_df
```

	Launch Site	Lat	Long
0	CCAFS LC-40	28.562302	-80.577356
1	CCAFS SLC-40	28.563197	-80.576820
2	KSC LC-39A	28.573255	-80.646895
3	VAFB SLC-4E	34.632834	-120.610745

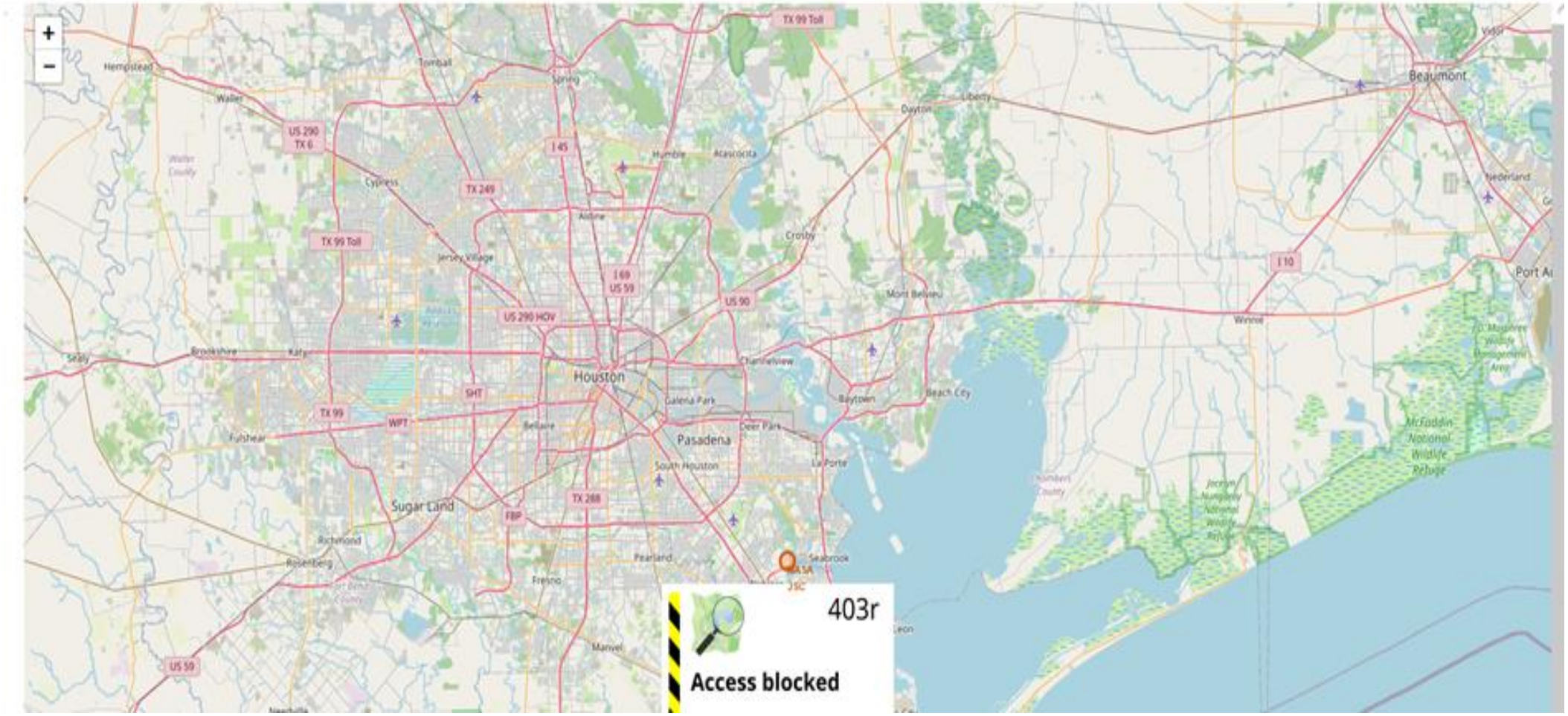
ANALYSE VISUELLE_CARTE FOLIUM

```
from folium.features import DivIcon

# Create a blue circle at NASA Johnson Space Center's coordinate with a popup label showing its name
circle = folium.Circle(nasa_coordinate, radius=1000, color='#d35400', fill=True).add_child(folium.Popup('NASA Johnson Space Center'))
# Create a blue circle at NASA Johnson Space Center's coordinate with a icon showing its name
marker = folium.map.Marker(
    nasa_coordinate,
    # Create an icon as a text label
    icon=DivIcon(
        icon_size=(20,20),
        icon_anchor=(0,0),
        html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' % 'NASA JSC',
        )
    )
site_map.add_child(circle)
site_map.add_child(marker)
```

ANALYSE VISUELLE_CARTE FOLIUM

site_map

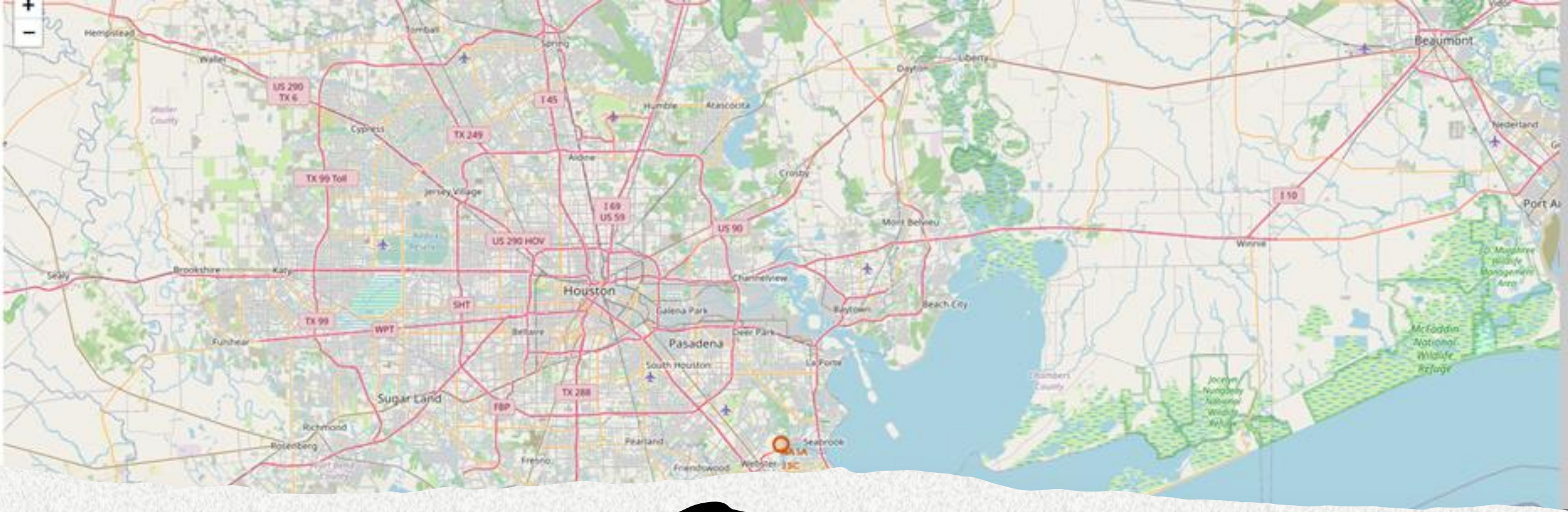


ANALYSE VISUELLE_CARTE FOLIUM

```
# Function to assign color to launch outcome
def assign_marker_color(launch_outcome):
    if launch_outcome == 1:
        return 'green'
    else:
        return 'red'

spacex_df['marker_color'] = spacex_df['class'].apply(assign_marker_color)
spacex_df.tail(10)
```

	Launch Site	Lat	Long	class	marker_color
46	KSC LC-39A	28.573255	-80.646895	1	green
47	KSC LC-39A	28.573255	-80.646895	1	green
48	KSC LC-39A	28.573255	-80.646895	1	green
49	CCAFS SLC-40	28.563197	-80.576820	1	green
50	CCAFS SLC-40	28.563197	-80.576820	1	green
51	CCAFS SLC-40	28.563197	-80.576820	0	red
52	CCAFS SLC-40	28.563197	-80.576820	0	red
53	CCAFS SLC-40	28.563197	-80.576820	0	red
54	CCAFS SLC-40	28.563197	-80.576820	1	green
55	CCAFS SLC-40	28.563197	-80.576820	0	red

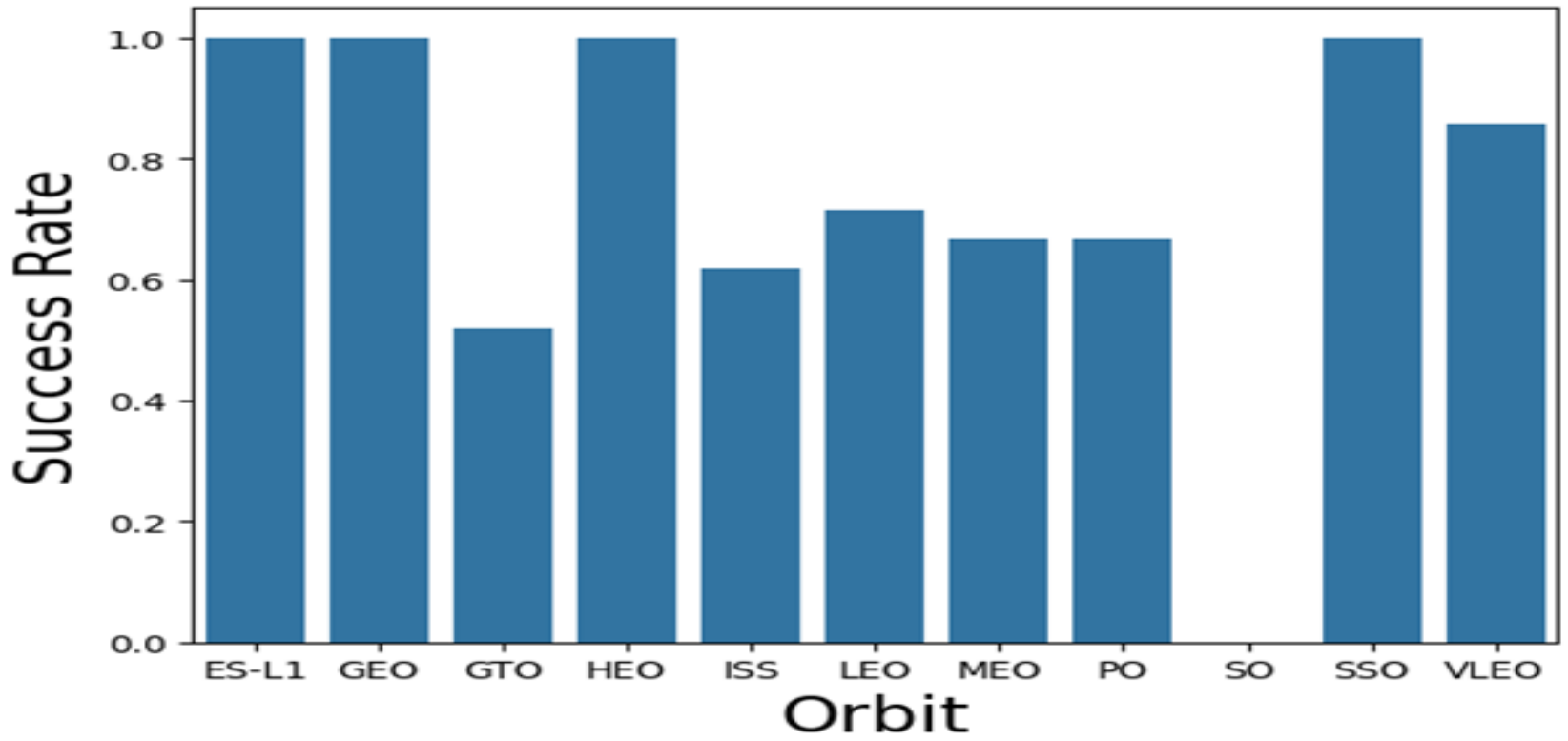


ANALYSE VISUELLE_CARTE FOLIUM

Analyse visuelle interactive

PLOTY DASH

```
orbit_success_rate = df.groupby('Orbit')['Class'].mean().reset_index()
sns.barplot(x='Orbit', y='Class', data=orbit_success_rate)
plt.xlabel("Orbit",fontsize=20)
plt.ylabel("Success Rate",fontsize=20)
plt.show()
```



PREDICTVE ANALYSIS

```
print('Accuracy on test data: ', logreg_cv.score(X_test, Y_test))
```

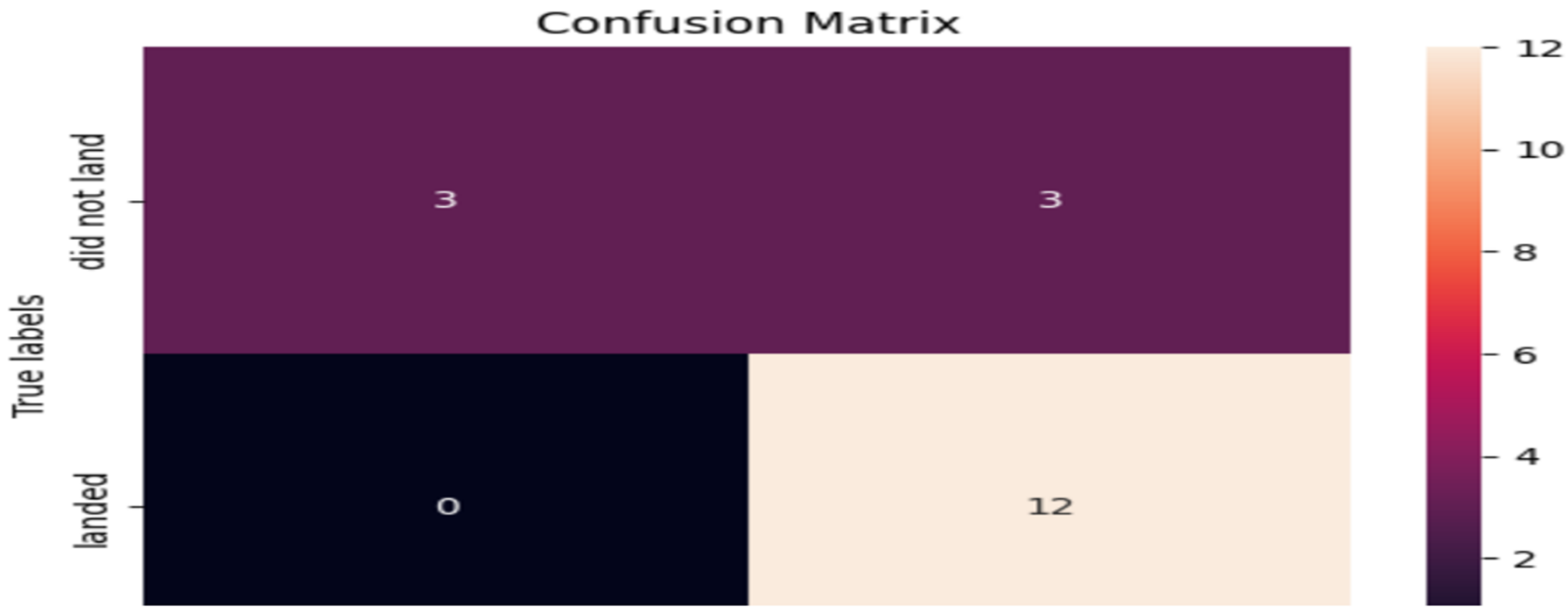
```
Accuracy on test data:  0.8333333333333334
```

```
print('Accuracy on test data: ', logreg_cv.score(X_test, Y_test))
```

```
Accuracy on test data:  0.8333333333333334
```

```
yhat = logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```

```
yhat = logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test, yhat)
```



ANALYSE PREDICTIVE (CLASSIFICATION)

TASK 6

Create a support vector machine object then create a `GridSearchCV` object `svm_cv` with cv = 10. Fit the object to find the best parameters from the dictionary `parameters`.

```
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
              'C': np.logspace(-3, 3, 5),
              'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
svm_cv = GridSearchCV(svm, parameters, cv=10)
svm_cv.fit(X_train, Y_train)
print("tuned hyperparameters :(best parameters) ", svm_cv.best_params_)
print("accuracy :", svm_cv.best_score_)
```

```
tuned hyperparameters :(best parameters)  {'C': np.float64(1.0), 'gamma': np.float64(0.03162277660168379), 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

ANALYSE PREDICTIVE (CLASSIFICATION)

```
print('Accuracy on test data: ', svm_cv.score(X_test, Y_test))
```

```
Accuracy on test data:  0.8333333333333334
```

Calculate the accuracy on the test data using the method `score`:

```
print('Accuracy on test data for SVM: ', svm_cv.score(X_test, Y_test))
```

```
Accuracy on test data for SVM:  0.8333333333333334
```

We can plot the confusion matrix

```
yhat=svm_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

ANALYSE PREDICTIVE (CLASSIFICATION)

TASK 8

Create a decision tree classifier object then create a `GridSearchCV` object `tree_cv` with parameters from the dictionary `parameters`.

```
parameters = {'criterion': ['gini', 'entropy'],
      'splitter': ['best', 'random'],
      'max_depth': [2*n for n in range(1,10)],
      'max_features': ['auto', 'sqrt'],
      'min_samples_leaf': [1, 2, 4],
      'min_samples_split': [2, 5, 10]}

tree = DecisionTreeClassifier()
tree_cv = GridSearchCV(tree, parameters, cv=10)
tree_cv.fit(X_train, Y_train)
print("tuned hyperparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

```
0.80535714 0.79107143 0.80357143 0.7625       0.7625       0.80535714
0.81964286 0.775        0.70892857 0.73214286 0.7875       0.76428571
         nan          nan          nan          nan          nan          nan
         nan          nan          nan          nan          nan          nan
         nan          nan          nan          nan          nan          nan
0.80357143 0.77857143 0.77678571 0.77678571 0.7625       0.78214286
0.76428571 0.7625       0.74821429 0.78214286 0.76071429 0.86071429
0.76607143 0.73571429 0.82142857 0.80357143 0.76428571 0.7625
         nan          nan          nan          nan          nan          nan
         nan          nan          nan          nan          nan          nan
         nan          nan          nan          nan          nan          nan
0.81964286 0.7625       0.80714286 0.84821429 0.84642857 0.77857143
0.775        0.84821429 0.72142857 0.82142857 0.77678571 0.81964286
0.81785714 0.83392857 0.76071429 0.775        0.7625       0.81071429
         nan          nan          nan          nan          nan          nan
         nan          nan          nan          nan          nan          nan
         nan          nan          nan          nan          nan          nan
0.81785714 0.83214286 0.72142857 0.80535714 0.79285714 0.81964286
0.83214286 0.79107143 0.76428571 0.80535714 0.775        0.80535714
```

ANALYSE PREDICTIVE (CLASSIFICATION)

TASK 10

Create a k nearest neighbors object then create a `GridSearchCV` object `knn_cv` with cv = 10. Fit the object from the dictionary `parameters`.

```
parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
              'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
              'p': [1,2]}

KNN = KNeighborsClassifier()
knn_cv = GridSearchCV(KNN, parameters, cv=10)
knn_cv.fit(X_train, Y_train)
print("tuned hyperparameters :(best parameters) ",knn_cv.best_params_)
print("accuracy :",knn_cv.best_score_)
```

```
tuned hyperparameters :(best parameters)  {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}
accuracy : 0.8482142857142858
```

ANALYSE PREDICTIVE (CLASSIFICATION)

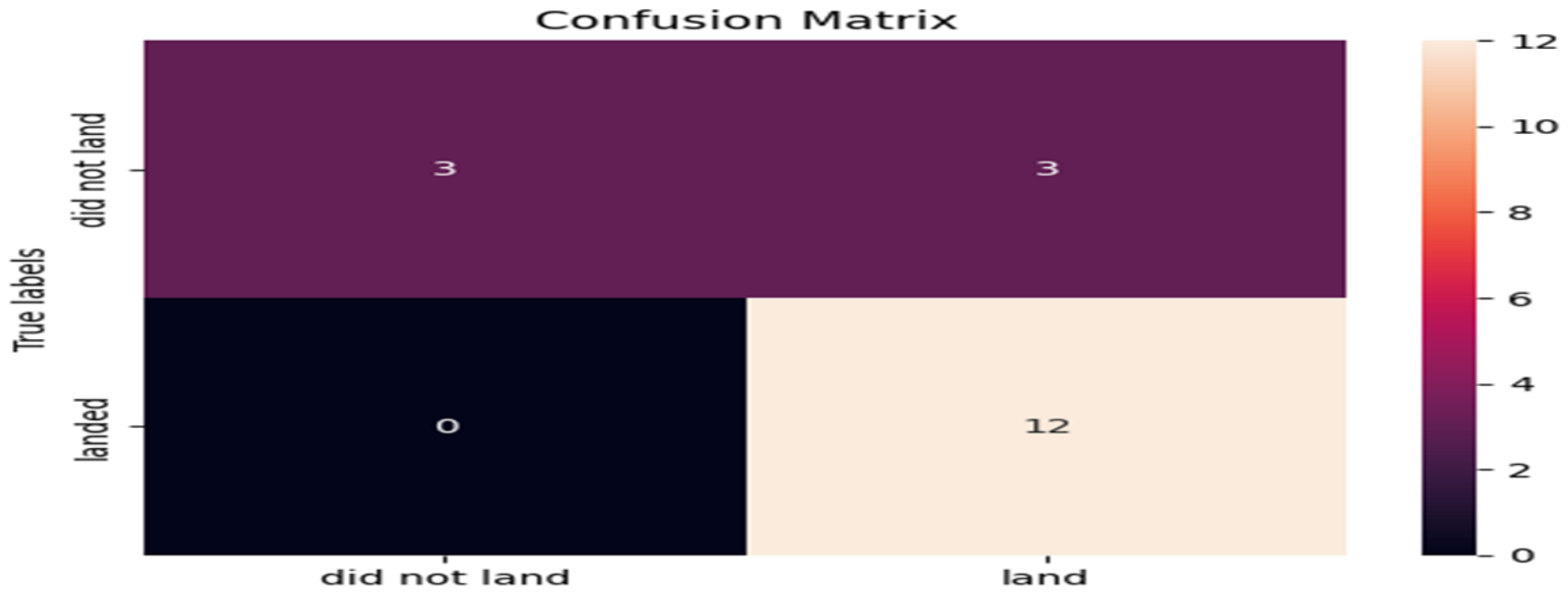
```
print('Accuracy on test data for KNN: ', knn_cv.score(X_test, Y_test))
```

```
Accuracy on test data for KNN:  0.8333333333333334
```

```
Start coding or generate with AI.
```

We can plot the confusion matrix

```
yhat = knn_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```



CONCLUSION

Ce rapport retrace les étapes de la collecte et de la préparation des données. Les outils comme python, SQL et Folium nous ont permis de traiter les données et de sortir les résultats qui sont consignés dans ce présent rapport.